```
Find products whose MRP is greater than the average MRP of all
products.

Display:

ProductID --> product
ProductName --> product
CategoryName --> category
MRP --> product

Sort from highest MRP to lowest.



select avg(MRP)
from `e1.products`



select
  p.ProductID,
  p.ProductName,
  c.CategoryName,
  p.MRP
from `e1.products` as p
inner join `e1.categories` as c
on p.CategoryID = c.CategoryID
where p.MRP > (
                select avg(MRP)
                from `e1.products`
              )
order by p.MRP desc
```

```
For each warehouse, calculate total stock and classify the
warehouse as:


High Stock → total stock >= 5,000
Medium Stock → total stock >= 2,500
Low Stock → total stock < 2,500

Display:

WarehouseID --> warehouse
Warehouse name --> warehouse
Total stock --> inventory
Average stock --> inventory
Stock classification --> case when on stock column --> inventory

Sort warehouses by total stock descending.


select
  w.WarehouseID,
  w.Warehouse,
  sum(i.Stock) as total_stock,
  round(avg(i.Stock),2) as avg_stock,
  case
```

```
    when sum(i.Stock) >= 5000 then "High Stock"
    when sum(i.Stock) >= 2500 then "Medium Stock"
    else "Low Stock"
  end as StockClassification
from `e1.warehouses` as w
inner join `e1.inventory` as i
on w.WarehouseID = i.WarehouseID
group by
  w.WarehouseID,
  w.Warehouse
order by total_stock desc



Find customers who have placed orders on more than 2 different
order dates.

Display:

CustomerID --> customers
CustomerName --> customers
Number of different order dates --> orders
Total number of orders --> orders
Total quantity purchased --> order_items

Sort by number of different order dates from highest to lowest.
--> distinct order_dates



select
```

```
  c.CustomerID,
  c.Name,
  count(distinct o.OrderDate) as no_of_orderdates,
  count(distinct o.OrderID) as no_of_orders,
  sum(oi.Quantity) as total_qty
from `e1.customers` as c
inner join `e1.orders` as o
  on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
  on o.OrderID = oi.OrderID
group by
  c.CustomerID,
  c.Name
having no_of_orderdates > 2
order by no_of_orderdates desc


For each category, find the product having the highest MRP.

Display:

CategoryName --> category
ProductID --> product
ProductName --> product
MRP --> product

Sort categories alphabetically.


select
```

```
  c.CategoryName,
  p.ProductID,
  p.ProductName,
  p.MRP
from `e1.products` as p
inner join `e1.categories` as c
on p.CategoryID = c.CategoryID
where p.MRP = (
                select max(p2.MRP)
                from `e1.products` as p2
                where p2.CategoryID = p.CategoryID
              )
order by c.CategoryName
```

Total sales made by each product?

```
select
  ProductID,
  sum(Total) as total_sales
from `e1.order_items`
group by ProductID
```

Show every order-item record, but also show the total sales of that product beside every row.

```
select
```

```
ProductID
from `e1.order_items`



select
  ProductID,
  OrderID,
  Total,
  sum(Total) over(partition by ProductID) as product_total_sales
from `e1.order_items`



-- function() over(partition by column order by column)

For every order-item record, display:

ProductID
SellingPrice
Average selling price of that product



select
  ProductID,
  SellingPrice,
  avg(SellingPrice) over(partition by ProductID) as
prd_avg_price
from `e1.order_items`
```

```
For every order-item record, show:

ProductID
OrderID
Number of order-item records for that product


select
  ProductID,
  OrderID,
  count(*) over(partition by ProductID) as no_of_records
  from `e1.order_items`




For every order-item record, display:

OrderID
ProductID
Total
Total sales across the entire order_items table

select
  orderID,
  OrderID,
```

```
  ProductID,
  Total,
  overall_sales,
  round((total*100)/overall_sales, 4) as perc_total_sales
from (select
  OrderID,
  ProductID,
  Total,
  sum(Total) over() as overall_sales
from `e1.order_items`)



Display every order-item record and assign a row number based on
Total, from highest to lowest.

select
  OrderID,
  ProductID,
  Total,
  row_number() over(order by total desc) as row_num
from `e1.order_items`
```